

Финальный prompt для Replit Agent

Главная цель приложения

Создай MVP-приложение для КЛИМ, цель которого — **сделать закупку сырья управляемой, быстрой и наглядной на основе реальных утренних остатков и плановых рецептов.**

Приложение не должно быть просто загрузчиком PDF и не должно быть просто красивой оболочкой над таблицей.

Его цель:

- автоматически забирать фактические остатки сырья из PDF по Полоцку;
- получать остатки по Липковской;
- учитывать сырьё в пути;
- извлекать потребность из PDF-рецептов;
- записывать всё это в Google Sheets;
- запускать обновление расчётной картины;
- показывать пользователю, **что нужно закупать, что можно закрыть переброской и где уже есть риск дефицита.**

Иначе говоря, приложение должно превращать разрозненные входы:

- PDF остатков,
- данные по складам,
- сырьё в пути,
- PDF рецептов,

в один понятный результат для закупщика и руководителя:

- текущая доступность сырья;
- ожидаемый остаток после плана;
- статус по каждой позиции;
- объём закупки;
- объём возможной переброски.

Роль Google Sheets

Google Sheets остаётся главным расчётным ядром MVP.

То есть приложение не должно переносить в backend всю бизнес-логику расчёта Decisions на первом этапе.

Приложение должно:

- получать и нормализовать входные данные;
- записывать их в нужные листы Google Sheets;
- пересчитывать уже рассчитанный лист Decisions;
- показывать пользователю итоговую управленческую картину.

Главный UX-принцип

Вся ежедневная работа должна выполняться с **одного экрана**.

Не нужно делать многоэкранную систему с переходами между Files, Recipes, Review, Input, Decisions.

Нужен один рабочий экран оператора, где доступны все ключевые действия дня.

Что должно быть доступно с одного экрана

На одной странице пользователь должен иметь возможность:

1. загрузить PDF с остатками Полоцка;
2. загрузить PDF рецепта;
3. вручную внести данные по Липковской;
4. вручную внести сырьё в пути;
5. увидеть статус обработки файлов;
6. увидеть строки, требующие ручной проверки;
7. сразу увидеть итоговую таблицу Decisions.

Основной бизнес-сценарий

Сценарий 1. Остатки Полоцка

1. Кладовщик загружает PDF с остатками Полоцка.
2. Приложение извлекает из PDF строки с сырьём и остатками.
3. Приложение сопоставляет сырьё со справочником Raw_Materials.
4. Распознанные строки записываются в Plant_Stock.
5. Нераспознанные или сомнительные строки попадают в блок ручной проверки.
6. После записи приложение пересчитывает Decisions.
7. Пользователь сразу видит обновлённые статусы и потребность в закупке.

Сценарий 2. Липковская

1. Пользователь на этом же экране вводит данные по Липковской.
2. Можно вводить общий остаток и резерв или сразу свободный остаток.
3. Если свободный остаток не введён, приложение считает его автоматически.
4. Данные записываются в `Lipkovskaya_Stock`.
5. После этого перечитывается `Decisions`.

Сценарий 3. Сырьё в пути

1. Пользователь вручную вводит сырьё в пути.
2. Указывает количество, ETA, направление, статус и комментарий.
3. Приложение пишет данные в `Inbound`.
4. После сохранения перечитывается `Decisions`.

Сценарий 4. Рецепт

1. Технолог загружает PDF рецепта.
2. Приложение извлекает шапку рецепта и строки состава.
3. Записывает шапку в `Recipes`.
4. Записывает строки состава в `Recipe_Lines`.
5. Формирует потребность по сырью и записывает её в `Need`.
6. Если есть сомнительные строки, они попадают в блок ручной проверки.
7. После этого перечитывается `Decisions`.

Что пользователь должен получить в итоге

После любого действия на едином экране пользователь должен видеть не просто факт загрузки файла, а **управленческий результат**:

- какие позиции в норме;
- какие позиции на контроле;
- какие позиции нужно закупать;
- какие позиции срочно к закупке;
- сколько можно закрыть Липковской;
- сколько нужно купить.

Главный смысл приложения — не хранить файлы, а **помогать принимать решение по закупке сырья**.

Структура единого экрана

Сделай один основной экран со следующими блоками:

Блок А. Загрузка файлов

Две большие кнопки:

- Загрузить PDF остатков Полоцка
- Загрузить PDF рецепта

Рядом показывать:

- имя файла;
- статус обработки;
- количество извлечённых строк;
- количество строк, отправленных на review.

Блок В. Ручной ввод

Форма Липковская

Поля:

- дата;
- сырьё;
- общий остаток;
- резерв;
- свободный остаток.

Форма В пути

Поля:

- сырьё;
- количество;
- ETA;
- направление;
- статус;
- комментарий / документ.

Блок С. Требуется проверки

Показывать строки из остатков и рецептов, которые не сопоставились автоматически.

Пользователь должен прямо на этом экране иметь возможность:

- выбрать нужный raw_uid;
- подтвердить строку;
- отправить в исключения.

Блок D. Управленческие решения

Показывать таблицу из Decisions со столбцами:

- код сырья;
- наименование;
- среднемесячный расход;
- порог закупки;
- остаток Полоцк;
- свободно Липковская;
- в пути;
- плановая потребность;
- доступно до плана;
- ожидаемый остаток после плана;
- статус;
- переброска с Липковской;
- закупка.

Обязательно:

- поиск;
- фильтр по статусу;
- сортировка по закупке;
- цветовая индикация статусов.

Google Sheets листы, с которыми нужно работать

Приложение должно читать/писать следующие листы:

- Raw_Materials
- Plant_Stock
- Lipkovskaya_Stock
- Inbound

- Recipes
- Recipe_Lines
- Need
- Decisions
- File_Registry
- App_Log

Что нужно реализовать в backend

Используй:

- Python
- FastAPI
- Google Drive API
- Google Sheets API или gspread
- pdfplumber / pymupdf
- OCR для сканированных PDF
- pandas для нормализации

Backend должен уметь:

- принимать загруженные PDF;
- извлекать текст и табличные данные;
- сопоставлять сырьё со справочником;
- писать данные в нужные листы Google Sheets;
- фиксировать ошибки и статусы обработки;
- перечитывать Decisions после каждого изменения;
- отдавать все данные на один главный экран.

Что нужно реализовать в frontend

Сделай простой, строгий, деловой интерфейс.

Требования:

- один основной экран;
- крупные рабочие кнопки;
- минимум кликов;
- понятные статусы;
- быстрый просмотр проблемных позиций;
- обновление данных без переходов по страницам.

Допустимы отдельные технические страницы:

- login
- settings
- logs

Но вся ежедневная работа должна происходить на одном экране.

Логи и контроль

Нужно логировать:

- загрузку файла;
- старт обработки;
- успешный парсинг;
- ошибки;
- строки на review;
- запись в Google Sheets;
- ручные исправления пользователя.

Логи писать в App_Log.

Реестр файлов вести в File_Registry.

Что пока не делать

На первом этапе не делать:

- сложную ролевую систему;
- 1С интеграцию;
- Telegram-бота;
- перенос бизнес-логики из Google Sheets в отдельную БД;
- AI-прогноз закупки;
- мобильное приложение.

Критерии готовности MVP

MVP готов, если:

1. Пользователь с одного экрана загружает PDF остатков Полоцка.
2. Пользователь с того же экрана загружает PDF рецепта.
3. Пользователь с того же экрана вручную вносит Липковскую.
4. Пользователь с того же экрана вручную вносит сырьё в пути.
5. Приложение пишет данные в Google Sheets.

6. Приложение формирует Recipes, Recipe_Lines, Need, Plant_Stock, Lipkovskaya_Stock, Inbound.
7. Приложение показывает строки, требующие проверки.
8. После любого действия приложение перечитывает Decisions.
9. На том же экране пользователь видит актуальную картину по закупке.

Финальная команда

Собери MVP-приложение для КЛМ, цель которого — превратить PDF остатков, данные по складам, сырьё в пути и PDF рецептов в единый ежедневный инструмент принятия решений по закупке сырья. Интерфейс должен быть построен по принципу одного рабочего экрана. Google Sheets остаётся расчётным ядром. Приложение должно извлекать данные из PDF, записывать их в нужные листы, показывать спорные строки для ручной проверки и сразу отображать актуальный лист Decisions как главный управленческий результат.